Analyse Technique Détaillée de `main.py`

Le script `main.py` implémente un pipeline complet de vision par ordinateur (Computer Vision) et d'analyse d'images visant à extraire, de manière totalement non supervisée, des séries temporelles (ou coordonnées XY) à partir de graphiques statiques matriciels (images PNG, JPG).

L'architecture du script repose sur **quatre grandes étapes séquentielles** dont les résultats se nourrissent mutuellement :

1. L'Optical Character Recognition (OCR) et la calibration affine des axes.
2. L'extraction de la légende par analyse morphologique et colorimétrique.
3. La segmentation non supervisée des courbes colorées et la soustraction morphologique pour la courbe noire.
4. La projection inverse et la vectorisation de la transformation pixellaire.

Voici l'explication technique approfondie de chacun des algorithmes employés.

---

1. Moteur OCR et Calibration Affine (Consensus RANSAC)

1.1. Prétraitement et OCR (`run_ocr`, `parse_number`)

Les chiffres des axes sont souvent rendus à de faibles résolutions. Les modèles OCR standards (comme EasyOCR) peinent sur des polices de petite taille. - **Agrandissement (Upscaling) :** L'image subit une interpolation bicubique (`cv2.INTER_CUBIC`) avec un facteur `OCR_UPSCALE = 4`, contraint par la limite interne du réseau de neurones d'EasyOCR (`2560px`). - **Binarisation :** L'image est convertie en niveaux de gris et binarisée (seuil de 150). Cela maximise le contraste des caractères face aux artefacts de compression JPEG ou aux grilles de fond clair. - **Parsing tolérant :** Le résultat texte est parsé en gérant les caractères parasites fréquents (remplacement des virgules par des points, conversion du tiret typographique unicode `\u2212` en signe mathématique `-`, et retrait des caractères comme `_` ou `|` souvent lus à tort quand un trait de graduation touche un chiffre).

1.2. Alignement Spatial (`group_aligned`)

Pour distinguer les graduations de l'axe des abscisses (X) de celles de l'axe des ordonnées (Y), le code exploite la géométrie spatiale : - Les étiquettes de l'axe X partagent toutes approximativement la même ordonnée pixellaire (`cy` constant). - Les

étiquettes de l'axe Y partagent toutes approximativement la même abscisse pixellaire (`cx` constant). Une fonction de regroupement avec tolérance agglomère les chiffres par proximité d'axe.

1.3. Modèle Linéaire Robuste - Algorithme RANSAC (`fit_axis`)

Le problème majeur de l'OCR est l'occurrence de faux positifs (ex: lire un `5` au lieu d'un `0`). Une simple régression linéaire par moindres carrés (`np.polyfit`) serait biaisée irrémédiablement par un seul point aberrant. Le script implémente une variante du **Random Sample Consensus (RANSAC)** appliquée à la totalité des paires possibles : 1. Pour **chaque paire** d'étiquettes (i, j), on calcule les coefficients de la transformation affine reliant l'espace des pixels à l'espace des réels : $$a = \frac{V_j - V_i}{P_j - P_i}$$ $$b = V_i - a \cdot P_i$$ 2. On applique cette droite hypothétique à *tous* les autres points détectés. 3. On compte les *inliers*, c'est-à-dire les points dont la valeur prédite est à une distance inférieure à une tolérance ϵ (fixée à 4% de l'étendue des valeurs lues) de la valeur lue par l'OCR. 4. La droite retenue est celle qui maximise le nombre d'inliers. 5. Si ce sous-ensemble d'inliers possède au moins 3 valeurs (avec vérification supplémentaire de régularité d'espacement via `evenly_spaced` pour discriminer les faux accords), on recalcule une droite finale par régression linéaire (`np.polyfit`) sur l'ensemble robuste. Sinon, on refuse la calibration et on active le **Mode Sauvetage (Pixels)**.

---

2. Analyse Sémantique : Association des Légendes (`extract_legends`)

Pour extraire automatiquement les noms des courbes, le code recherche une association géométrique entre les chaînes de caractères (non numériques) et des échantillons de couleurs.

1. **Extraction de la ROI (Region of Interest) :** Pour chaque boîte de texte détectée, le script scrute une petite zone située immédiatement à sa gauche (là où se trouvent classiquement les marqueurs de légende).
2. **Filtrage Saturation/Valeur :** Dans l'espace HSV, les pixels achromatiques (texte noir, fond blanc) sont ignorés par seuillage (`S > 50` ou `V < 160`).
3. **Analyse de Composantes Connexes (`cv2.connectedComponentsWithStats`) :** Le script évalue la morphologie des objets colorés trouvés :
4. Un marqueur valide doit être plein (`fill_ratio >= 0.6`).
5. Il doit respecter les proportions d'un carré (pastille) ou d'un rectangle allongé (ligne).
6. Les lettres isolées (faible taux de remplissage) ou les poussières (petite aire) sont rejetées.

7. **Appariement :** La couleur RGB médiane de ce marqueur est sauvegardée. Plus tard, un appariement par distance euclidienne minimale (voisin le plus proche sous seuil) lie cette couleur à l'une des courbes extraites.

---

3. Segmentation des Courbes (Clustering & Morphologie)

C'est le cœur analytique de l'algorithme : extraire les signaux visuels des traces du graphe. Il est divisé en deux moteurs : un pour les couleurs, un pour le tracé noir.

3.1. Approche Non Supervisée pour les Courbes Colorées (`detect_curves`)

Les courbes colorées subissent des modifications RGB dues à l'anti-aliasing et à la compression. Travailler en RGB pur échoue. - **Espace HSV (Teinte, Saturation, Valeur) :** Le script isole les pixels significativement colorés (`S > 50` et `V > 50`). - **Topologie Circulaire de la Teinte :** La composante `Hue` est un angle de 0 à 180° dans OpenCV. Le problème est que 179° (rouge) et 1° (rouge) sont distants numériquement mais adjacents visuellement. Le code mappe la teinte sur le cercle trigonométrique unitaire : $$X_{feat} = \cos(H_{rad}) \times 100$$ $$Y_{feat} = \sin(H_{rad}) \times 100$$ - **Clustering MeanShift :** Plutôt que de contraindre l'algorithme à un nombre fixe de k-clusters (comme le ferait KMeans), le script utilise `MeanShift`. C'est un algorithme basé sur l'estimation par noyau de la densité (Kernel Density Estimation). Il fait « glisser » des fenêtres vers les maxima locaux de densité (modes) sur la variété mathématique, permettant de découvrir **automatiquement** le nombre de couleurs distinctes (donc le nombre de courbes).

3.2. Soustraction Morphologique pour la Courbe Noire (`extract_black_curves`)

Extraire le noir est complexe car le graphe est envahi d'éléments noirs structurels : texte, bordures de l'image, axes, et grille (si grise foncée). L'approche est soustractive : 1. **Seuillage binaire inversé :** On garde tous les pixels sombres de l'image. 2. **Gommage spatial du texte :** Les `bounding_boxes` du texte OCR sont remplies de noir (effacement pur et simple). 3. **Morphologie Mathématique (Lignes) :** Le code utilise `cv2.morphologyEx` avec l'opération `MORPH_OPEN`. L'ouverture (érosion suivie de dilatation) est effectuée avec des éléments structurants asymétriques géants : un rectangle `(40, 1)` et un `(1, 40)`. Cela filtre tout ce qui n'est pas un trait parfaitement droit d'au moins 40 pixels de long (les axes et la grille). Ces éléments reconnus sont soustraits de l'image. 4. **Inhibition des Bords (Anti-aliasing) :** Les courbes colorées ont des bords assombris. Si on ne les enlève pas, l'algorithme détecterait des "courbes noires" fantômes collées aux courbes bleues ou rouges. Le masque des courbes colorées est

donc dilaté (`cv2.dilate` sur 3×3 pixels) et soustrait de l'image noire. 5. **Nettoyage topologique (`cv2.findContours`) :** Les petits artefacts résiduels (restes de lettres, petites poussières) sont filtrés par un critère de longueur d'arc (`arcLength > 20`). L'utilisation de l'arc plutôt que l'aire (`contourArea`) est cruciale car une ligne mathématique de 1 pixel d'épaisseur a une aire de 0.

---

4. Re-projection spatiale et Export Vectoriel (`column_mean`, `to_real`)

Une fois qu'une courbe est identifiée sous forme de nuage de pixels (en coordonnées Y_{px}, X_{px}), il faut extraire une fonction $f(x) = y$.

4.1. Extraction du signal (Calcul vectorisé)

Dans une image, une ligne épaisse s'étale sur plusieurs pixels en y pour un x donné. L'algorithme doit calculer la moyenne des y pour chaque x. L'implémentation utilise l'algèbre matricielle pure pour la performance (sans boucle Python) :

```
ux, inv = np.unique(xs, return_inverse=True)
mean_y = np.bincount(inv, weights=ys) / np.bincount(inv)
```

- `np.unique` regroupe toutes les colonnes x.
- `np.bincount` additionne instantanément (en C optimisé) la somme des Y par colonne X, et le nombre de pixels. La division matricielle produit un signal 1D net.

4.2. Projection dans le domaine réel

Le vecteur X_{px} est transformé via l'équation de la droite affine obtenue par RANSAC (Étape 1). $$X_{real} = V_1 + \frac{(X_{px} - P_1) \cdot (V_2 - V_1)}{P_2 - P_1}$$ Si une des échelles n'a pas pu être calibrée, les coordonnées restent à l'échelle des pixels d'origine, évitant la production de données factices.

4.3. Exportation

Enfin, via la bibliothèque Pandas, les données (re-mappées spatialement, triées sur l'axe X, et séparées par noms de légende) sont fusionnées (`pd.concat`) sous forme tabulaire et exportées au format Excel natif (`.xlsx`), accompagnées d'un plot de vérification (`matplotlib`).